axol

How the recommendation engine works — the exact algorithm, as implemented

Prepared November 09, 2026. Source of truth: `server/src` in github.com/lukeb2222/axol. Every formula below is the code's behaviour, not an aspiration.

In one paragraph

axol keeps one **interest vector** per person: a list of topic tags with signed weights. Every clip carries its own tags with relevance scores. When a clip is served, the app measures how long the person actually watched it, whether they looped, liked, shared, hid or skipped it, and converts that into a single **engagement score**. That score nudges the weight of every tag on the clip (fast loop, after every clip). Every twelve clips a language model reads the recent history and the weights, rewrites the person's taste summary, adjusts weights semantically, and issues new discovery queries (slow loop). Ranking combines interest fit, a formal exploration bonus, freshness, platform affinity, diversity, and one deliberate wild card per batch.

1. Data model

Entity	Fields that matter	Purpose
interests (per user)	tag, weight ∈ [−5, 5], impressions, positive, origin ∈ {onboarding, learned, ai}	The person's taste. Weights are the sum of visible nudges; nothing is a black box.
videos	source, external_id, tags_json = [{tag, relevance ∈ [0,1]}], duration_ms	One row per clip across YouTube Shorts, TikTok, Instagram Reels.
impressions	watched_ms, duration_ms, replays, liked, shared, hidden, skipped, score, reason	One row per serving; accumulates behaviour, then holds the final score.
source_affinity	source, views, score_sum	Mean engagement per platform, used as a ranking bonus.
discovery_queries	source, query, last_run_at	What to search for on each platform; refreshed by re-synthesis.

2. Cold start: onboarding → profile

Input: free text ("I'm obsessed with skateboarding and jazz piano… no politics") plus optional tapped tags. A single structured call to Claude (schema-enforced JSON) returns:

- **interests**: {tag, weight 0–1, why} — central interests, weighted by how central they are.
- **adjacent**: tasteful guesses the person did not mention, weights 0.2–0.5, to widen the feed from day one.

- **avoid**: only explicit dislikes. These are hard filters at ranking time, never soft penalties.
- **queries**: 6–10 discovery queries per platform, mixing formats (tutorial, POV, highlights, comedy) within the topics.
- **summary**: the "what the feed thinks of you" paragraph shown on the profile screen.

Onboarding weights are scaled ×2.5 (a top interest starts at 2.5 of a maximum 5) and adjacent ones ×1.5, leaving headroom for behaviour to overtake stated preference. Without an AI key, a 90-topic keyword taxonomy with prefix matching extracts tags and a small grammar extracts dislikes; the rest of the system is identical.

3. Ingestion and tagging

Each (user, platform, query) is a discovery query. Discovery runs at onboarding, after each re-synthesis, and lazily whenever the person's pool of unseen clips drops below 12, bounded to 12 seconds so the feed never stalls.

- **YouTube Shorts**: official Data API — search.list with videoDuration=short and videoEmbeddable=true, then videos.list for durations; clips over 3 minutes are dropped.
- **TikTok / Reels**: a pluggable discovery provider that returns URLs, or links the person pastes. Public oEmbed enriches titles and creators.

New clips are tagged in batches of 25 by one structured Claude call at low effort, producing 3–8 kebab-case tags each with relevance scores. Tags cover topic, format, and vibe so they are reused across many clips.

4. What the app measures while you watch

Signal	How it is measured
watched_ms	YouTube: the IFrame Player is polled every 250 ms and only time in the PLAYING state counts. TikTok/IG embeds do not expose playback, so it is dwell time while the card is the active slide and the tab is visible.
duration_ms	From the player, from the Data API, or assumed 20 s for opaque embeds.
replays	Each ENDED event; clips are looped manually by seeking to 0.
liked / shared / hidden	Explicit taps. Double-tap is a like.
skipped	The card left the screen with under 2.5 s watched.

Progress is sent every 2 s (survives tab close via keepalive). A `finish` event fires when the card scrolls away; learning runs once per impression on finish, skip, or hide.

5. Engagement score

```
completion = min(1.5, watched_ms / max(duration_ms, 5000))      # loops push it above 1
if hidden:                                    score = -1.5
elif watched < 2.5 s and completion < 0.25: score = -1.0        # hard skip
else:                                         score = clamp((completion - 0.45) * 2, -1, 1.2)
score += 0.6 if liked, 0.5 if shared, 0.2 * min(3, replays)
score  = clamp(score, -1.5, 2)
```

The design choice is the **neutral point at 45 % completion**: watching less than half is negative, finishing is strongly positive, and looping or liking stacks on top. The dominant input is time, which is far harder to fake than a tap.

6. Interest update (fast loop, every clip)

```
decay:  every weight of this user *= 0.995                          # stays plastic, old interests fade
for each tag t on the clip, relevance r, current weight w:
    room = 1 - min(1, |w| / 5)                                      # soft ceiling
    dw   = 0.18 * score * r * (0.35 + 0.65 * room)
    w'   = clamp(w + dw, -5, 5)
new tag (never seen before) is adopted only if score > 0.3 and r >= 0.5,
    starting at 0.18 * score * r                                    # the profile grows beyond what was sai
impressions[t] += 1;  positive[t] += 1 if score > 0.15
source_affinity[platform] += score
```

This is an online, stochastic-gradient style update on a linear preference model: cheap, incremental, and fully explainable. Every weight is literally the sum of nudges the user could watch happen on screen.

7. Ranking

Candidates are all tagged clips the user has not been served, minus any carrying an avoided tag. Each gets:

```
match     = sum_t( w_t * r_t ) / max(1, sum_t r_t)                 # interest fit
explore   = 0.35 * sum_t( r_t * sqrt( ln(N + 2) / (n_t + 1) ) ) / max(1, sum_t r_t)
            # UCB1 bonus: tags shown rarely (small n_t) earn a trial; N = user's total views
freshness = 0.20 * exp(-age_days / 14)
platform  = 0.30 * source_affinity[source]
noise     = 0.08 * Gumbel()                                         # stochastic ordering
total     = match + explore + freshness + platform + noise
```

Batch assembly is greedy with a diversity penalty: after each pick, any candidate sharing one of its top-3 tags loses 0.3 × relevance per repeat and the same creator loses 0.45 per repeat. Then, with 60 % probability, the highest-explore leftover is inserted at position 4 as a **wild card** labelled "Something new, to see if it lands". That is the variable-reward element: most of the batch is reliably on-taste, one card is a genuine surprise, and its outcome feeds straight back into step 6.

Each card carries a human-readable reason ("Because you watch skateboarding") derived from the largest $w_t \cdot r_t$ term.

8. Re-synthesis (slow loop, every 12 clips)

The server sends Claude the current weights table plus the last 40 clips with their tags, watch percentage, likes, loops and skips. The structured response contains:

- **adjustments**: {tag, delta ∈ [−1, 1], reason} → applied as w += 1.5 · delta.
- **new_interests**: exploratory interests introduced at low weight (origin = ai).
- **queries**: fresh discovery queries leaning into what is working → triggers ingestion.
- **summary** and a one-line **insight** ("You keep watching jazz piano to the end…") shown as a toast.

The numeric learner handles fast, per-clip adaptation. The language model handles the slower semantic layer: noticing that "sourdough" and "wood-fired pizza" are one appetite, that the skateboarding interest is really about filmmaking, or that a stated interest is being skipped every time.

9. Safeguards and user control

- Avoided tags are hard filters; users can boost, mute, or remove any interest from the profile screen.
- Weights saturate softly and decay globally, so no single interest can permanently dominate the feed.
- Every re-synthesis is logged and visible to the user as tuning history, with the model's one-line reason.
- All AI calls have deterministic fallbacks; an outage or refusal degrades quality, never availability.
- Playback uses each platform's official embed player. No video is downloaded or rehosted.

10. Parameters at a glance

Parameter	Value	Effect
Learning rate	0.18	Step size of each nudge
Max weight	5	Soft ceiling via the room term
Decay per event	0.995	Old interests fade over ~140 clips
Neutral completion	45 %	Below = negative, above = positive
Quick-skip threshold	2.5 s	Hard skip → score −1
Re-synthesis cadence	12 clips	Slow-loop frequency
Explore weight	0.35	UCB bonus scale
Freshness weight / half-life	0.20 / 14 days	Newer clips get a small lift
Diversity penalty	0.30 per repeat	Keeps batches varied
Wild card probability	60 %	One exploration card at position 4
Min unseen pool	12	Triggers discovery